

# 1. Introduzione al progetto

MicroBot OS è un sistema operativo sperimentale sviluppato con l'obiettivo di esplorare in modo concreto la costruzione di un ambiente software completo a basso livello, partendo da zero e senza l'utilizzo di librerie o framework esterni. Il progetto nasce dall'esigenza di comprendere in profondità il funzionamento interno di un sistema operativo, andando oltre l'utilizzo di strumenti ad alto livello e affrontando direttamente la gestione dell'hardware, del rendering grafico e dell'interazione utente.

A differenza di molti progetti accademici che si limitano a dimostrazioni isolate, MicroBot OS si configura come una piattaforma integrata in cui ogni componente contribuisce a formare un sistema coerente. Il kernel gestisce direttamente il framebuffer per il rendering grafico, implementa un sistema di input da tastiera, integra un motore di rendering testuale e coordina diversi moduli logici, tra cui un sistema di eventi, una shell interattiva e un modello di controllo per entità denominate "MicroBot".

Il progetto non è stato concepito come un semplice esercizio tecnico, ma come una base reale per la costruzione di sistemi più complessi. In particolare, MicroBot OS rappresenta il primo passo verso la realizzazione di un ambiente capace di controllare e visualizzare il comportamento di un insieme di unità modulari, ispirate al concetto di swarm robotics. Questo approccio consente di collegare direttamente il livello software, rappresentato dall'interfaccia e dalla logica di controllo, con una possibile evoluzione fisica del sistema basata su micro-dispositivi reali.

Un elemento distintivo del progetto è la realizzazione di un'interfaccia grafica completamente custom, costruita direttamente sopra il framebuffer. L'intero sistema visivo, inclusi layout, pannelli, componenti e gerarchie informative, è stato progettato e implementato manualmente, con l'obiettivo di ottenere un'interfaccia chiara, leggibile e coerente. Questo approccio consente non solo un controllo totale sull'aspetto visivo, ma anche una comprensione profonda del processo di rendering e delle sue implicazioni a livello di performance.

MicroBot OS si colloca quindi a metà strada tra un sistema operativo didattico e una piattaforma sperimentale, unendo aspetti di ingegneria del software, programmazione di basso livello e progettazione di interfacce. Il progetto dimostra come sia possibile costruire, passo dopo passo, un sistema complesso partendo da componenti fondamentali, mantenendo al tempo stesso una visione chiara dell'evoluzione futura.

In prospettiva, questa base può essere estesa per includere funzionalità avanzate come comunicazione tra dispositivi, integrazione con hardware embedded (come ESP32), controllo remoto e interfacce immersive. In questo senso, MicroBot OS non rappresenta un punto di arrivo, ma l'inizio di un percorso più ampio verso la realizzazione di un ecosistema software e hardware integrato.

### 3. Architettura del sistema

L'architettura di MicroBot OS è stata progettata seguendo un approccio modulare, in cui ogni componente ha una responsabilità ben definita e interagisce con gli altri attraverso interfacce semplici e controllate. Questo permette al sistema di rimanere comprensibile, estendibile e facilmente modificabile, anche durante le fasi evolutive del progetto.

Il sistema si sviluppa a partire dal processo di boot, gestito tramite GRUB e lo standard Multiboot2, che consente al kernel di ricevere informazioni fondamentali sull'hardware, in particolare riguardo al framebuffer. Una volta avviato, il kernel inizializza i sottosistemi principali e prende il controllo completo dell'ambiente di esecuzione, senza dipendere da sistemi operativi sottostanti.

Dal punto di vista strutturale, l'architettura può essere suddivisa in diversi livelli logici, ognuno dei quali contribuisce al funzionamento complessivo del sistema. I livelli non sono rigidamente separati come in sistemi complessi industriali, ma seguono una gerarchia chiara che riflette il flusso di esecuzione reale.

#### Struttura generale del sistema

| Livello   | Componente                | Descrizione                                              |
|-----------|---------------------------|----------------------------------------------------------|
| Boot      | GRUB + Multiboot2         | Caricamento del kernel e passaggio informazioni hardware |
| Core      | Kernel                    | Gestione centrale del sistema e coordinamento moduli     |
| Rendering | Framebuffer + Backbuffer  | Gestione grafica a basso livello                         |
| UI        | GFX Text + Layout         | Rendering testo e costruzione interfaccia                |
| Input     | Keyboard                  | Lettura input utente                                     |
| Logica    | MicroBot + Events + Shell | Comportamento sistema e interazione                      |

|      |           |                                           |
|------|-----------|-------------------------------------------|
| Loop | Main Loop | Ciclo continuo di input → update → render |
|------|-----------|-------------------------------------------|

L'esecuzione del sistema segue un flusso deterministico. Dopo l'inizializzazione del framebuffer e dei moduli principali, il kernel entra in un ciclo infinito in cui attende input dall'utente, aggiorna lo stato interno e ridisegna l'interfaccia. Questo modello, semplice ma efficace, consente di mantenere il controllo completo su ogni fase dell'esecuzione.

Flusso di esecuzione

| Fase | Operazione        | Descrizione                             |
|------|-------------------|-----------------------------------------|
| 1    | Bootloader        | GRUB carica il kernel                   |
| 2    | Parsing Multiboot | Lettura informazioni framebuffer        |
| 3    | Init Framebuffer  | Setup memoria video                     |
| 4    | Init Moduli       | Inizializzazione sottosistemi           |
| 5    | Boot Sequence     | Eventuale animazione di avvio           |
| 6    | Primo Render      | Disegno iniziale UI                     |
| 7    | Main Loop         | Gestione continua input e aggiornamenti |

Un elemento centrale dell'architettura è la separazione tra rendering e logica. Il sistema non utilizza un motore grafico esterno, ma implementa direttamente le operazioni di disegno, come riempimento di rettangoli, rendering di testo e gestione dei colori. Questo approccio consente di mantenere il controllo totale sulle prestazioni e sul comportamento visivo.

Il framebuffer viene utilizzato insieme a un backbuffer, che permette di evitare problemi di flickering durante il rendering. Tutte le operazioni grafiche vengono eseguite nel buffer secondario e poi copiate in un'unica operazione sullo schermo, garantendo una visualizzazione fluida e stabile.

---

### Componenti principali

| Modulo        | File          | Responsabilità         |
|---------------|---------------|------------------------|
| Kernel        | kernel.c      | Coordinamento generale |
| Framebuffer   | framebuffer.c | Disegno pixel e buffer |
| Text Renderer | gfx_text.c    | Rendering caratteri    |
| Input         | keyboard.c    | Lettura tastiera       |
| Events        | events.c      | Log eventi sistema     |
| MicroBot      | microbot.c    | Gestione entità        |
| Shell         | shell.c       | Interfaccia testuale   |

---

Dal punto di vista della comunicazione interna, i moduli non sono completamente isolati, ma condividono uno stato globale controllato. Questo è tipico dei sistemi a basso livello, dove la semplicità e l'efficienza hanno priorità rispetto all'astrazione estrema. Tuttavia, la struttura è comunque organizzata in modo tale da permettere una futura evoluzione verso modelli più avanzati, come sistemi a messaggi o architetture event-driven più complesse.

Un altro aspetto importante è la gestione dello stato del sistema. Informazioni come la schermata corrente, lo stato dei MicroBot, il buffer della shell e il log degli eventi vengono mantenute e aggiornate continuamente. L'interfaccia grafica non fa altro che rappresentare visivamente questo stato, creando un legame diretto tra logica interna e output visivo.

Infine, l'architettura è stata pensata per essere estesa. Ogni modulo può essere migliorato o sostituito senza dover riscrivere completamente il sistema. Questo rende MicroBot OS una base solida per sviluppi futuri, come l'integrazione con hardware reale, l'introduzione di networking o la costruzione di interfacce più avanzate.